Week 8: Database Relationship Analysis using Joins

Name	Akash L
Enroll No.	TU6253210211001
Week	8
Topic	Database Relationship Analysis using Joins

1. Create and combine the tables

```
CREATE TABLE Customer (
    customer_id INT PRIMARY KEY,
    name        VARCHAR(50),
    city        VARCHAR(50)
);
CREATE TABLE Product (
    product_id  INT PRIMARY KEY,
    prod_name   VARCHAR(50),
    price       DECIMAL(10,2)
);
CREATE TABLE Orders (
    order_id    INT PRIMARY KEY,
    customer_id INT,
    product_id  INT,
    quantity    INT,
    order_date  DATE,
    FOREIGN KEY (customer_id) REFERENCES Customer(customer_id),
    FOREIGN KEY (product_id)  REFERENCES Product(product_id)
);
CREATE TABLE Payment (
    payment_id  INT PRIMARY KEY,
    order_id    INT,
    amount      DECIMAL(10,2),
    method      VARCHAR(20),
    status      VARCHAR(10),
    FOREIGN KEY (order_id) REFERENCES Orders(order_id)
);
```

Sample data

```
INSERT INTO Customer VALUES
(1,'Arun','Chennai'),(2,'Divya','Madurai'),
(3,'Karthik','Coimbatore'),(4,'Meena','Trichy');
INSERT INTO Product VALUES
(1,'Laptop',55000),(2,'Mouse',500),
(3,'Keyboard',1200),(4,'Headset',2000);
INSERT INTO Orders VALUES
(101,1,1,1,'2026-10-01'),(102,2,2,2,'2026-10-03'),
(103,1,3,1,'2026-10-05'),(104,3,2,1,'2026-10-07'),
(105,2,3,2,'2026-10-09');
INSERT INTO Payment VALUES
(1,101,55000,'UPI','Paid'),(2,102,1000,'Card','Paid'),
(3,103,1200,'Cash','Paid'),(4,104,500,'UPI','Paid');
```

Meena has no orders, the Headset was never ordered, and order 105 has no payment. These gaps show the difference between the join types.

2. Implement the joins

INNER JOIN

Returns only matching rows. Order 105 is missing because it has no payment.

```
SELECT c.name, o.order_id, p.method
FROM Customer c
INNER JOIN Orders o  ON c.customer_id = o.customer_id
INNER JOIN Payment p ON o.order_id    = p.order_id;
```

name	order_id	method
Arun	101	UPI
Divya	102	Card
Arun	103	Cash
Karthik	104	UPI

LEFT JOIN

Returns all customers, even those with no orders.

```
SELECT c.name, o.order_id
FROM Customer c
LEFT JOIN Orders o ON c.customer_id = o.customer_id;
```

name	order_id
Arun	101
Arun	103
Divya	102
Divya	105
Karthik	104
Meena	NULL

RIGHT JOIN

Returns all products, even those never ordered.

```
SELECT pr.prod_name, o.order_id
FROM Orders o
RIGHT JOIN Product pr ON o.product_id = pr.product_id;
```

prod_name	order_id
Laptop	101
Mouse	102
Mouse	104
Keyboard	103
Keyboard	105
Headset	NULL

3. Retrieve complete order details

```
SELECT o.order_id, c.name, pr.prod_name, o.quantity,
       (o.quantity * pr.price) AS total_amount, p.method, p.status
FROM Orders o
JOIN Customer c     ON o.customer_id = c.customer_id
JOIN Product pr     ON o.product_id  = pr.product_id
LEFT JOIN Payment p ON o.order_id    = p.order_id;
```

order_id	name	prod_name	qty	total_amount	method	status
101	Arun	Laptop	1	55000	UPI	Paid
102	Divya	Mouse	2	1000	Card	Paid
103	Arun	Keyboard	1	1200	Cash	Paid
104	Karthik	Mouse	1	500	UPI	Paid
105	Divya	Keyboard	2	2400	NULL	NULL

4. Display customer purchase history

```
SELECT c.name, o.order_id, o.order_date, pr.prod_name,
       o.quantity, COALESCE(p.status,'Unpaid') AS pay_status
FROM Customer c
LEFT JOIN Orders o   ON c.customer_id = o.customer_id
LEFT JOIN Product pr ON o.product_id  = pr.product_id
LEFT JOIN Payment p  ON o.order_id    = p.order_id
ORDER BY c.name, o.order_date;
```

name	order_id	order_date	prod_name	qty	pay_status
Arun	101	2026-10-01	Laptop	1	Paid
Arun	103	2026-10-05	Keyboard	1	Paid
Divya	102	2026-10-03	Mouse	2	Paid
Divya	105	2026-10-09	Keyboard	2	Unpaid
Karthik	104	2026-10-07	Mouse	1	Paid
Meena	NULL	NULL	NULL	NULL	Unpaid

5. Generate multi-table reports

Report A: total spending per customer

```
SELECT c.name, COUNT(o.order_id) AS total_orders,
       COALESCE(SUM(o.quantity * pr.price),0) AS total_spent
FROM Customer c
LEFT JOIN Orders o   ON c.customer_id = o.customer_id
LEFT JOIN Product pr ON o.product_id  = pr.product_id
GROUP BY c.name;
```

name	total_orders	total_spent
Arun	2	56200
Divya	2	3400
Karthik	1	500
Meena	0	0

Report B: pending (unpaid) orders

```
SELECT o.order_id, c.name, pr.prod_name
FROM Orders o
JOIN Customer c ON o.customer_id = c.customer_id
JOIN Product pr ON o.product_id  = pr.product_id
LEFT JOIN Payment p ON o.order_id = p.order_id
WHERE p.payment_id IS NULL;
```

order_id	name	prod_name
105	Divya	Keyboard

Conclusion

INNER JOIN shows only matched records, LEFT JOIN keeps every row from the left table (customers without orders), and RIGHT JOIN keeps every row from the right table (products never ordered). Combining all four tables gives complete order details, purchase history, and summary reports.

Note: RIGHT JOIN works in MySQL and SQL Server but not in older SQLite versions. There, swap the table order and use LEFT JOIN.